Viktoriya Kurmisheva
Final Project Technical Documentation
ITAI 2373
Anna Devarakonda

NewsBot Intelligence System 2.0 Technical Documentation

Introduction

This technical documentation explains how my NewsBot 2.0 final project works and why I built it this way. The project expands the midterm NewsBot into a larger news analysis platform that can classify articles, discover topics, summarize text, analyze sentiment, extract entities, find similar articles, and support a simple conversational workflow. I designed the system to run in Google Colab because that makes it easier for the professor to open and test without a paid setup. The project uses the BBC News Classification dataset and keeps the workflow practical instead of depending on paid APIs or private keys.

System Architecture

The system starts with a configuration class that stores important settings such as article limits, TF-IDF feature counts, topic count, and summary length. The dataset loader finds or uploads the BBC zip file ("BBC News Classification"), extracts the training data, standardizes the column names, creates simple titles, and validates that the dataset meets the project requirements. After that, the preprocessing class cleans the text, removes noise, tokenizes words, removes stop words, and lemmatizes terms. This structure keeps the early steps organized because every later module depends on clean and consistent text.

Advanced Content Analysis Engine

The advanced classifier uses TF-IDF features, sentiment features, and article length features to predict the article category. I compared Complement Naive Bayes, Logistic Regression, and Linear SVM so the notebook shows more than one modeling option. Logistic Regression became the production model because it gives probabilities and supports explanation-style output, which makes the results easier to understand. The final saved notebook output shows about 97.99% accuracy and a macro F1 score around 97.91%, so the classifier performs strongly on the BBC test split.

Topic Modeling and Sentiment Analysis

The topic discovery module uses both LDA and NMF to find hidden themes in the article collection. LDA works with count-based features, while NMF works with TF-IDF features, so the notebook demonstrates two different topic modeling approaches. The system assigns every article to a dominant topic and then compares those discovered topics with the original BBC categories. The sentiment tracker uses VADER to label articles as positive, negative, or neutral and also creates category-level sentiment summaries that support trend-style analysis.

Entity Extraction and Relationships

The entity mapper uses spaCy to find people, organizations, places, dates, money amounts, products, and events. It also builds co-occurrence relationships by counting which entities appear

together in the same article. This matters because news analysis often depends on connections between companies, political groups, leaders, places, and events. The network graph is not meant to prove a relationship by itself, but it gives a useful first look at repeated names and connections inside the dataset.

Language Understanding and Generation

The summarizer uses an extractive method, which means it chooses important sentences from the article instead of generating new text with a paid language model. This choice keeps the notebook free, simple, and reproducible. The semantic search engine uses TF-IDF similarity to find articles related to a user query or a selected article. The content enhancer combines the classifier, summarizer, topic engine, sentiment tracker, and entity mapper into one readable analysis package for a single article.

Multilingual and Conversational Features

The multilingual processor detects the article language and shows a transparent translation workflow. Because I did not want to use paid translation APIs, the notebook uses manual/offline fallback examples for Spanish and French demonstrations. The conversational interface uses rule-based intent detection for commands like summarize an article, search a topic, show top entities, show sentiment, or show topics. This gives the project a conversational layer without pretending it is a full chatbot trained on massive dialogue data.

Web Application Frontend

The final notebook includes a Gradio frontend that runs inside Colab. A user can paste a headline and article text, click the analyze button, and see the predicted category, sentiment, summary, insights, category probabilities, entities, similar articles, and classification explanation. This frontend matters because it turns the notebook into something closer to a real product demo. The Gradio share link is temporary while the Colab runtime is active, so the notebook itself remains the permanent submission artifact.

Testing and Evaluation

The project includes testing and evaluation sections so the system does not only look complete but also proves that it runs. The evaluator reports classification accuracy, macro precision, macro recall, macro F1, topic coverage, integration test quality, and runtime. In the saved notebook output, the evaluation report shows a 1.0 test pass rate and an average runtime of about 0.08 seconds per article for the integrated test example. These results help show that the system works as a connected pipeline instead of a group of unrelated notebook cells.

Configuration and Setup

The setup process is designed for a normal Colab user. The first cells check for needed packages, install missing ones, download NLTK resources, and load the spaCy English model. The user then uploads the BBC dataset zip (“BBC News Classification”) when Colab asks for it. The most important configuration values are stored in one class, which makes it easier to adjust article limits, feature counts, topic counts, and summary length without searching through the whole notebook.

Limitations and Future Technical Work

The system has limits even though it works well for the class project. The BBC dataset does not include real publication dates ("BBC News Classification"), so the notebook uses generated analysis months only for demonstration. The multilingual workflow is not a production translation system, and the bias/framing analyzer uses transparent word lists instead of deep context understanding. In the future, I would add real news feeds, real article dates, stronger open-source summarization, saved model files, and more complete unit tests. Those improvements would move the project closer to a production news intelligence platform.

Conclusion

In conclusion, NewsBot 2.0 shows how many NLP techniques can work together inside one complete system. The technical value comes from the integration of classification, topic modeling, sentiment, named entities, summarization, search, multilingual processing, conversation, evaluation, and a web interface. The project stays realistic because it works in Colab and avoids paid services. Building it helped me understand that a strong NLP project needs clean data, organized components, clear outputs, testing, and honest limits.

References:

"BBC News Classification." Kaggle.Com, https://www.kaggle.com/competitions/learn-ai-bbc/data. Accessed 14 July 2026.